

CDM Review 2 Document: System Design

Concepts of Data Mining

1. Data Analytics Design

The Analytics Module scans the localized DuckDB news warehouse.

- **Objective:** Discover term correlations and categorize massive streams of unstructured text.
- **Algorithms:** KMeans (Clustering), Apriori (Association Rules for Keyword sequences), SVM (Supervised Classification on frozen targets).

2. High-Level System Architecture Design

The architecture follows a decoupled client-server paradigm.

- **Client:** HTML/Vanilla JS with Chart.js.
- **Server:** Flask REST API routing to `mining_engine.py` and `analytics_engine.py`.
- **Data Engine:** DuckDB acting as the embedded transactional data store, interacting with `pandas` and `scikit-learn` models.

3. Database / Warehouse Design

- **Engine:** DuckDB (OLAP-optimized).
- **Primary Schema `news_articles`:**
 - `id` (VARCHAR PK)
 - `title` (VARCHAR)
 - `content` (TEXT)
 - `source` (VARCHAR)
 - `category` (VARCHAR)
 - `published_at` (TIMESTAMP)
 - `url` (VARCHAR)
- Data is bulk-imported through CSV streams and incrementally updated via live APIs.

4. Screen / Input / Form Design

- **Main Interface:** A singular Dashboard (`index.html`).
- **Input:** Global Search Bar, Date Range Filters, Parameter Sliders for Algorithm Tuning (e.g., configuring K-clusters).
- **Forms:** Data upload forms allowing analysts to upload custom `.csv` datasets on the fly.

5. Visualization / Graph / Chart Design

Visualizations are generated on the frontend using Chart.js consuming stats endpoints:

- **Bar Charts:** Displaying Category Distribution.
- **Scatter Plots:** 2D PCA representation of K-Means clusters.
- **Line Graphs:** Temporal trending volume of specific news tags over time.
- **Tables:** Apriori Rules metrics (Confidence, Support, Lift).

6. Test Data and Test Case Design

6.1 Test Data

- `frozen_corpus.csv` (~120k records from AG News).
- Synthetic edge cases (null fields, malformed dates) for robust ingestion testing.

6.2 Test Cases

- **TC1:** Rebuilding Index with updated corpus size -> Expected: Index regenerates in < 15s.
- **TC2:** Running SVM on 10k sample -> Expected: Accuracy > 85%, F1 Score calculated.
- **TC3:** Dashboard rendering performance -> Expected: Renders 5 charts in under 2 seconds.

7. Issues Faced & Resolutions

During the architectural design and system implementation, several technical hurdles were encountered:

1. Massive Memory Overhead during Data Mining (K-Means/SVM):

- *Issue:* Running complex unsupervised clustering (K-Means) and Apriori on the entire 120,000-record dataset caused massive RAM consumption and timeouts.
- *Resolution:* Implemented dynamic bounds like `max_features` constraints in TF-IDF for the CDM pipeline to limit dimensionality, and established a "frozen_corpus" pattern to ensure reproducible, fast review metrics.

2. Index Synchronization Rebuild Loops:

- *Issue:* Whenever the backend restarted or the live-news fetcher triggered, the system rebuilt the *entire* index for all 120k articles. This caused the backend startup to halt for roughly an hour while new Vector Embeddings were mapped.
- *Resolution:* Developed an `update_index` incremental methodology where the backend compares the DuckDB `rowid` counts to the `index_meta.json`. It now uniquely bounds and encodes *only* freshly added articles to append to FAISS, dropping update time from 1 hour to `< 5 seconds`.

3. Module Import Failures on Refactor:

- *Issue:* Python's strict module pathing (`sys.path`) broke when separating the backend APIs from the CDM analytics and utility extraction scripts.
- *Resolution:* Built an absolute path resolution configuration using `os.path.abspath(__file__)` to allow terminal execution from any working directory across the project.